



Pontificia Universidad Católica de Chile  
IA Lab



**IA** Lab

## Case Study

Julio Hurtado & Felipe del Río

Computer Science Department, PUC

# Intro

Today we are going to review the course material in a practical way.

How?

- 1 Define a task
- 2 Place relevant questions for each step.
- 3 Solve it step-by-step answering these questions.
- 4 In the practice session, we are going to solve the problem hands on.

Shorter than usual, so you might have the chance to ask lot's of questions.

# Table of Contents

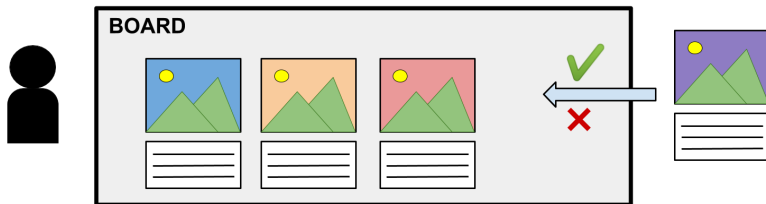
- 1 Intro
- 2 Problem
- 3 Data
- 4 Model
- 5 Double click in Data Representation
- 6 Metrics
- 7 Conclusions
- 8 Questions?
- 9 Appendix

# Problem

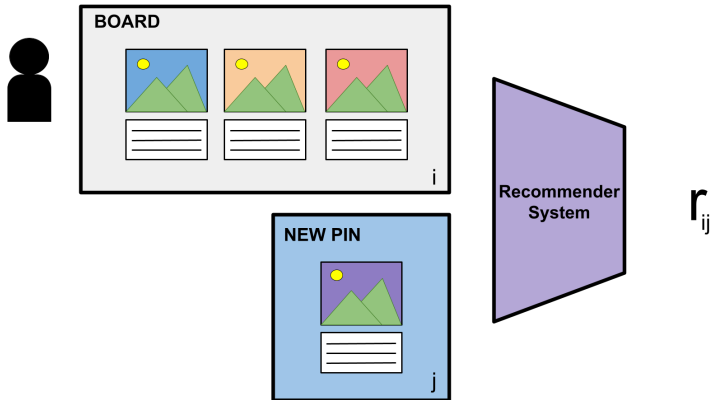
### Questions

- 1 What are we trying to solve?
- 2 How are we going to frame the problem?

- In this class we are going to make recommendations to users based on the image they've seen and comments the have written on them.
- We are going to base this recommendation based on previous users' interactions.



How can we frame this problem?

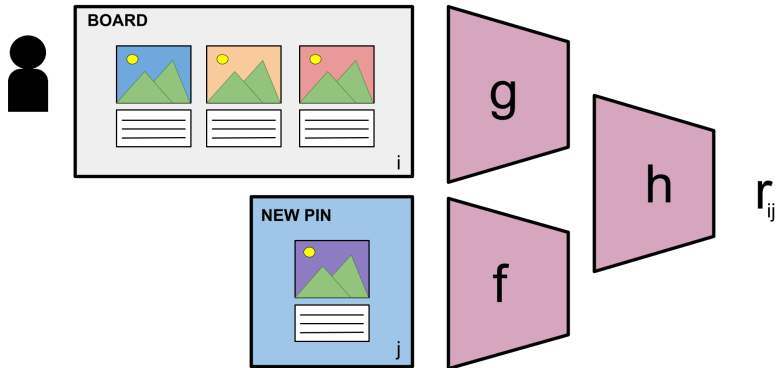


Given a user  $i$ ,  $u_i$ , and, a pin  $j$  given by an image  $x_j$  and a comment  $c_j$ .

- 1 For each pair user-pin we would like to compute a relevance score.

$$r_{ij} = h(g(u_i), f(x_j, c_j))$$

- 2 Then recommend the most relevant pins only to a particular user.



Given our relevance formula:

$$r_{ij} = h(g(u_i), f(x_j, c_j))$$

## What do we need?

- How do we represent a pin (image-comment)?  
We will learn a function ( $f$ ) to represent them using NNs.
- How do we represent a user?  
As the set of all his/her pins ( $g$ , trying to reuse  $f$ ).
- How do we measure relevance?  
Using a distance function ( $h$ ).

We'll dive deeper when we see the model...

# Data

## Questions

- 1 What data do we have available?
- 2 What info it contains and which kind it is?
- 3 How much data do we have available?
- 4 Based on the data available. Can a human solve the problem?<sup>a</sup>

---

<sup>a</sup> <https://hbr.org/2016/11/what-artificial-intelligence-can-and-cant-do-right-now>

We chose the **Pinterest dataset**<sup>1</sup> for this activity.

### Description

- 70.200 pins from 117 users in Pinterest.
- Each pin contains a short raw text and an image.
- Every image is already embedded using a CNN.

### Attention

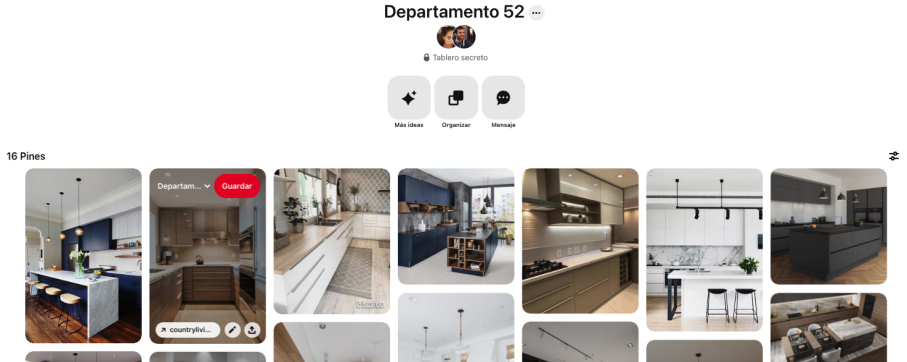
We are not going to use images but features already extracted from them.

For practical issues we're going to use a sub-sample, though.

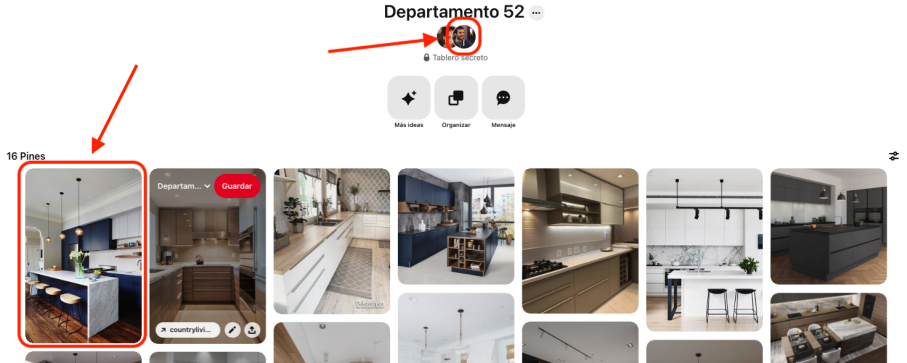
---

<sup>1</sup><https://data.mendeley.com/datasets/fs4k2zc5j5/3>

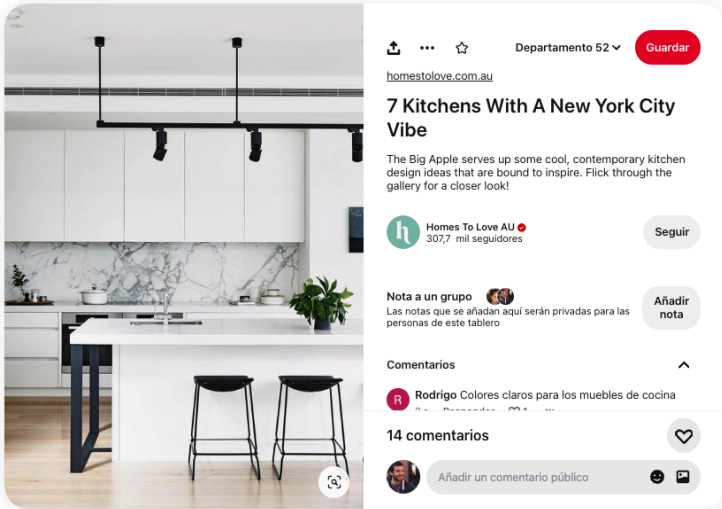
# Example of Pinterest Board



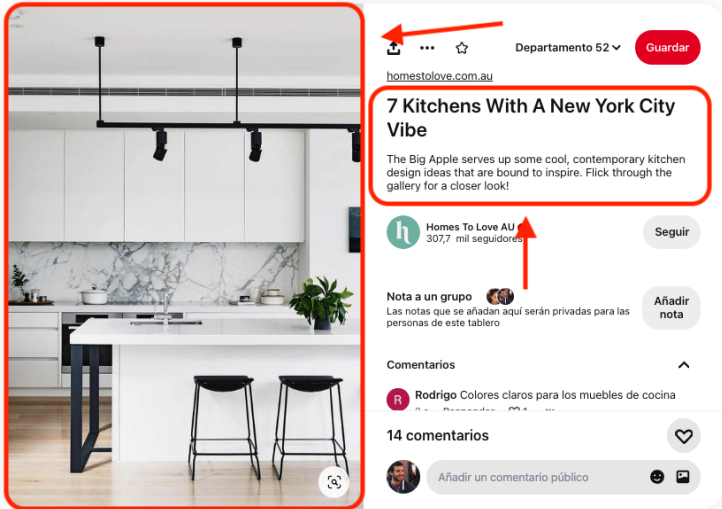
# Example of Pinterest Board



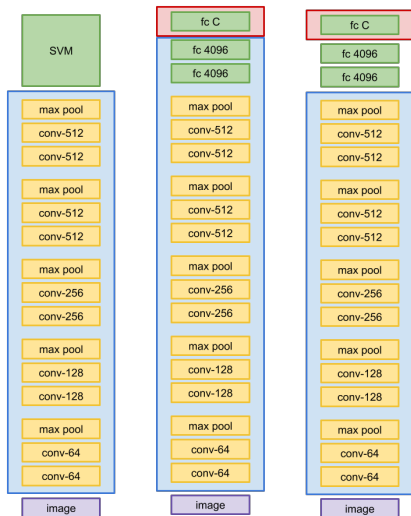
# Example of Pinterest Pin



# Example of Pinterest Pin



# Remember: Direct Transfer Learning



## Recipe

- Freeze the learned feature extracting layers.
- Replace and train a classifier over this features.
- Other types of classifiers, such as SVMs, can be used as well.

## Questions

- 1 What data do we have available?  
Pinterest dataset.
- 2 What info it contains and which kind it is?  
Text interactions of users with images.
- 3 How much data do we have available?  
70.200 pins from 117 users.
- 4 Based on the data available. An expert can solve the problem?  
Probably.

# Model

### Questions

- 1 What is the nature of the input and output data?
- 2 What kind of model are we going to use?
- 3 What kind of supervision are we going to use?
- 4 What kind of training are we going to use?

Given our relevance formula:

$$r_{ij} = h(g(u_i), f(x_j, c_j))$$

### What do we need?

- How do we represent a pin (image-comment)?  
We will learn a function ( $f$ ) to represent them using NNs.
- How do we represent a user?  
As the set of all his/her pins ( $g$ , trying to reuse  $f$ ).
- How do we measure relevance?  
Using a distance function ( $h$ ).

## Input

- Text → Transformers or RNNs
- Images → CNNs

## Output

- Recommendation Set → ?

## Input

- Text → Transformers or RNN
- Images → CNN or Transformer

## Output

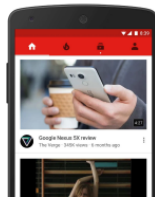
- User → Classification loss (cross entropy)

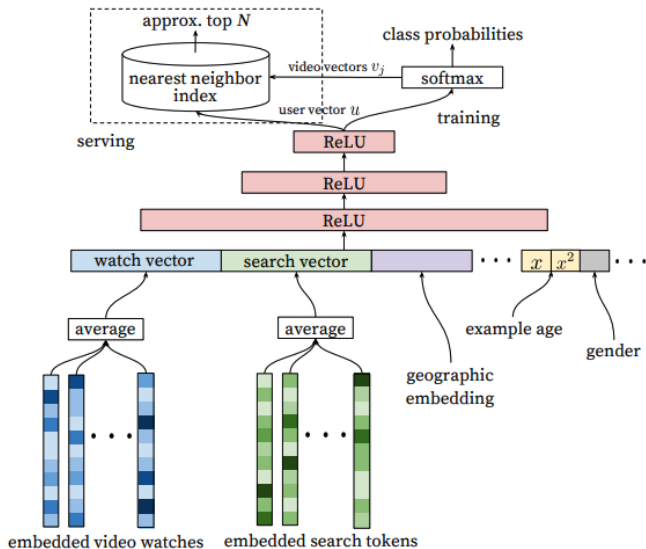
## Deep Neural Networks for YouTube Recommendations

Paul Covington, Jay Adams, Emre Sargin  
Google  
Mountain View, CA  
{pcovington, jka, msargin}@google.com

### ABSTRACT

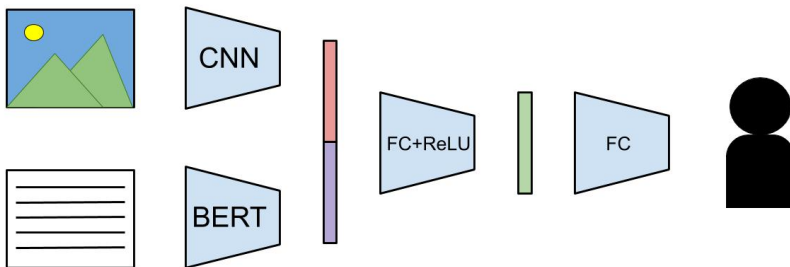
YouTube represents one of the largest scale and most sophisticated industrial recommendation systems in existence. In this paper, we describe the system at a high level and focus on the dramatic performance improvements brought by deep learning. The paper is split according to the classic two-stage information retrieval dichotomy: first, we detail a deep candidate generation model and then describe a separate deep ranking model. We also provide practical lessons and insights derived from designing, iterating and maintaining a massive recommendation system with enormous user-facing impact.



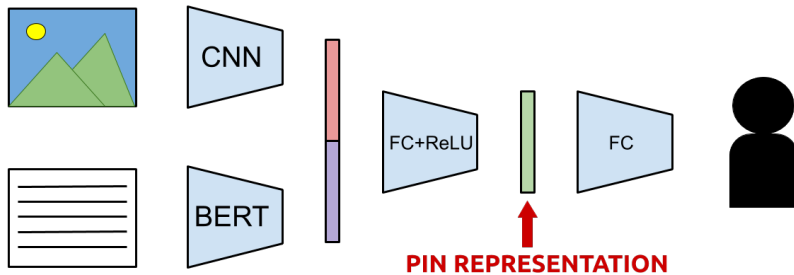


## Dropout

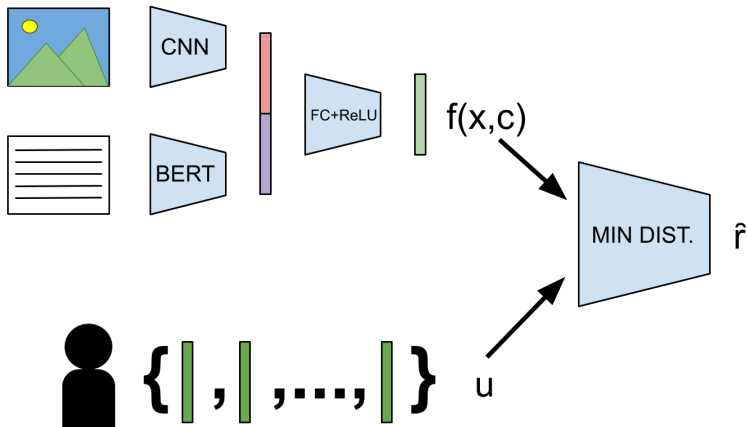
- On the image and text representations.
- After every fully connected layer (except last one).



# Architecture: Training

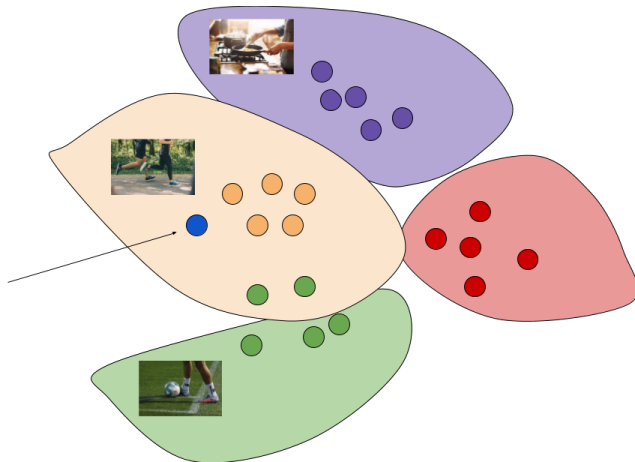


$$r_{ij} = h(g(u_i), f(x_j, c_j))$$

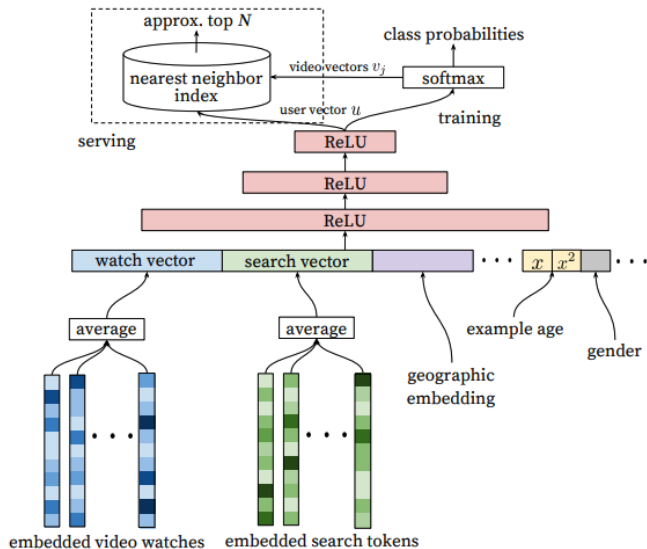


## Intuition

Items that correspond to the same user will end up closer together.

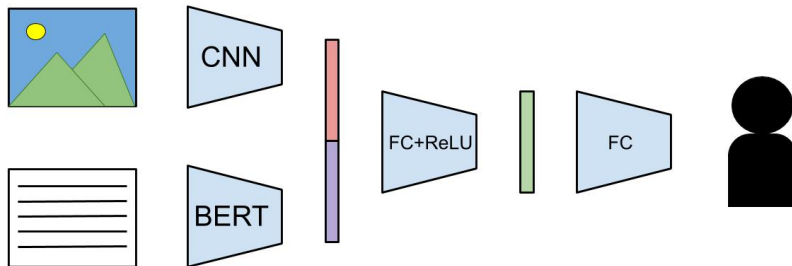


# Youtube Inference



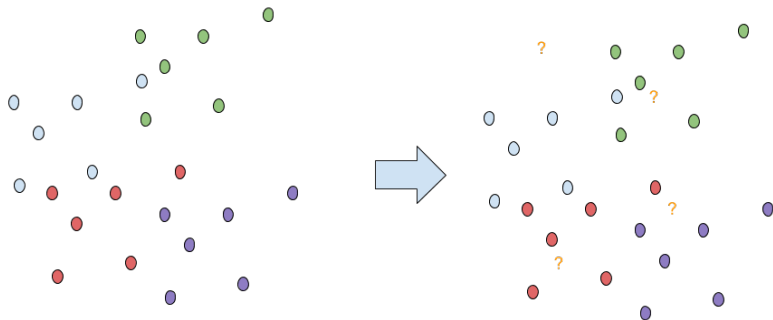
## Remember

Always pretrain as much as make sense!



## Recommendation Intuition

- Users prefer similar content to which they had interacted before.
- Images and text similar to which they already commented before.



## Double click in Data Representation

## Traditional (seen in DINTA)

- Text → Transformers or RNN
- Images → CNN or Transformer
- Sound → CNN or Transformer or RNN
- Video → CNN or Transformer (visual)  
+ RNN or Transformer (temporal)

## Others

- Categories → ?
- Properties → ?
- Products → ?
- Geographic Data → ?
- Etc... → ?

## Examples:

- Categories
- Tags
- Discrete properties
- Places
- Etc...

## Representation

- Embeddings (`torch.nn.Embedding`)
- Language model?

## Examples:

- Numeric or continuous properties
- Geo (latitud, longitud)
- Etc...

## Representation

- Normalized value + linear layer
- Usually linear layer is important to impose a higher dimension to the property.

## Examples:

- Sets of tags.
- Items in a game.
- Etc...

## Representation

- Transformer encoder
- + [CLS] if need a complete representation.
- Without positional encoding.

## Examples:

- Shopping cart.
- Hierarchy of categories.
- Etc...

## Representation

- RNNs (beware if sequences are long)
- Transformer encoder + positional encoding.
- + [CLS] if need a complete representation.

## Examples:

- E-commerce products.
- Multiple user information.
- Etc...

## How?

- Concatenating representations.
- Summing representations.
- Transformer encoder (+ pos. encoding if order is important).
- RNNs if order is important.
- Usually good idea to use a linear layer after.

# Metrics

### Questions

- ① How will we measure performance?
- ② Which metrics are we going to tracked?
- ③ Which data should we use to measure performance?
- ④ How will we determine success?

# Why do we care about metrics?

## Capture goals in a quantitative way

- Capture and track business goals.
- Helps coordinate effort towards an objective value.
- Where do we stand:
  - Comparison to baseline.
  - Comparison to target performance.
  - Comparison to past performance.

## Training Objective

- It is only a proxy for the real world.
- Helps for debugging, e.g. non decreasing training loss.

## Task relevant properties

- Measure how good is our recommendation list.
- Sparse feedback (unbalanced).

## Decision

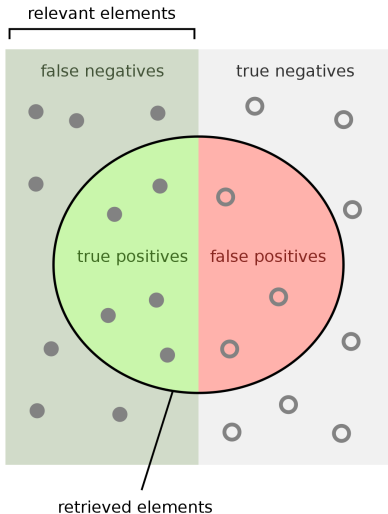
- Use Precision and Recall
- Use Ranking metrics

# Confusion Matrix

Not all errors are created equal.

|              |          | Predicted Class                            |                                                            |                                                          |
|--------------|----------|--------------------------------------------|------------------------------------------------------------|----------------------------------------------------------|
|              |          | Positive                                   | Negative                                                   |                                                          |
| Actual Class | Positive | True Positive (TP)                         | False Negative (FN)<br><b>Type II Error</b>                | <b>Sensitivity</b><br>$\frac{TP}{(TP + FN)}$             |
|              | Negative | False Positive (FP)<br><b>Type I Error</b> | True Negative (TN)                                         | <b>Specificity</b><br>$\frac{TN}{(TN + FP)}$             |
|              |          | <b>Precision</b><br>$\frac{TP}{(TP + FP)}$ | <b>Negative Predictive Value</b><br>$\frac{TN}{(TN + FN)}$ | <b>Accuracy</b><br>$\frac{TP + TN}{(TP + TN + FP + FN)}$ |

# Precision and Recall



How many retrieved items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are retrieved?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

|              |          | Predicted Class                            |                                                            |                                                          |
|--------------|----------|--------------------------------------------|------------------------------------------------------------|----------------------------------------------------------|
|              |          | Positive                                   | Negative                                                   |                                                          |
| Actual Class | Positive | True Positive (TP)                         | False Negative (FN)<br><b>Type II Error</b>                | <b>Sensitivity</b><br>$\frac{TP}{(TP + FN)}$             |
|              | Negative | False Positive (FP)<br><b>Type I Error</b> | True Negative (TN)                                         | <b>Specificity</b><br>$\frac{TN}{(TN + FP)}$             |
|              |          | <b>Precision</b><br>$\frac{TP}{(TP + FP)}$ | <b>Negative Predictive Value</b><br>$\frac{TN}{(TN + FN)}$ | <b>Accuracy</b><br>$\frac{TP + TN}{(TP + TN + FP + FN)}$ |

- We can use Precision@k and Recall@k.
- MAP (Mean Average Precision)
- MRR (Mean Reciprocal Rank)
- nDCG (Normalized Discounted Cumulative Gain)

## nDCG

$$DCG = \sum_i \frac{r_i}{\log_2(1 + i)}$$

$$nDCG = \frac{DCG}{iDCG}$$



And a total of 20 relevant items.

$$Recall@10 = \frac{5}{20} = 20\%$$

$$Precision@10 = \frac{5}{10} = 50\%$$

|              |                                                                                   |                                                                                   |                                                                                   |                                                                                   |                                                                                   |
|--------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|              | 1                                                                                 | 2                                                                                 | 3                                                                                 | 4                                                                                 | 5                                                                                 |
| Recomendador |  |  |  |  |  |

$$DCG = \frac{1}{\log_2(3)} + \frac{1}{\log_2(5)} + \frac{1}{\log_2(6)} = 1.4485$$

$$iDCG = \frac{1}{\log_2(2)} + \frac{1}{\log_2(3)} + \frac{1}{\log_2(4)} + \frac{1}{\log_2(5)} + \frac{1}{\log_2(6)} = 2.9485$$

$$nDCG = \frac{1.4485}{2.9485} = 0.4912$$

# Conclusions

- Propose a problem on which to apply known techniques.
- Propose questions to help us solve this problem.
- Follow this questions to propose a solution.
- Propose some metrics to evaluate our solution.

Questions?

# Appendix